

JAVA PROGRAMMING MASTERCLASS

Object 類別

「所有 Java 類別的共同祖先」

[← 返回目錄](#)

Outline

- 認識 **Object** 類別 — `java.lang.Object` 與繼承關係
- **Objects** 工具類別 — `java.util.Objects` 的 Null 安全設計
- 哈希碼與 `hashCode()` — `Objects.hash()` 的現代實作
- `equals()` 方法 — 搭配 Pattern Matching 的現代化寫法
- `toString()` 方法 — 物件的字串表示
- **Records** 與 **Object** 方法 — 自動實作 `toString`、`equals`、`hashCode`
- 其他 **Object** 方法 — `getClass()`、`clone()`、`finalize()`

認識 Object 類別

Object 類別是什麼？

- 位於 `java.lang` 套件，完整名稱 `java.lang.Object`
- 所有 Java 類別的父類別（根類別）
- 所有物件都隱含繼承了 Object 的 `public`、`protected` 方法
- 可依需要 **Override**（重新定義）這些方法

💡 提示：toString()、hashCode()、equals() 等方法都來自 Object 類別，不需要 import 即可使用

隱含繼承 Object 類別

所有類別都隱含地繼承 `Object`，以下兩種寫法完全相同：

```
1  class Animal {  
2      String name;  
3  }
```

```
1  class Animal extends Object {  
2      String name;  
3  }
```

Java 類別的繼承關係

常見類別皆以 **Object** 為父類別：

類別	父類別	說明
String	Object	字串類別
StringBuffer	Object	可變字串類別
Scanner	Object	輸入掃描器
自訂類別 (如 Animal)	Object	所有自訂類別

Object 常用方法

本章介紹下列四個方法：

方法	說明
<code>int hashCode()</code>	傳回物件的雜湊碼
<code>boolean equals(Object obj)</code>	比較兩個物件是否相同
<code>String toString()</code>	傳回代表物件的字串
<code>final Class getClass()</code>	傳回物件所屬的類別

Objects 工具類別 (JDK 7+)

`java.util.Objects` (注意：不是 `java.lang.Object`) 是 Java 7 引入的工具類別，提供靜態方法讓開發者更安全地處理 `null`。

方法	說明
<code>equals(a, b)</code>	比較 a, b 是否相等，自動處理 null
<code>hash(fields ...)</code>	根據傳入的欄位產生 hash 值
<code>requireNonNull(obj)</code>	檢查是否為 null，為空則拋出異常

Objects 工具類別 – 範例

```
1  import java.util.Objects;
2
3  String s1 = null;
4  String s2 = "Java";
5
6  // 傳統寫法會拋出 NPE (NullPointerException)
7  // System.out.println(s1.equals(s2));
8
9  // 安全寫法：若 s1 為 null 則回傳 false
10 System.out.println(Objects.equals(s1, s2)); // false
```

Objects.isNull() 與 nonNull()

方法

說明

`Objects.isNull(obj)`

回傳 `true` 若 `obj = null`，適合作為 Stream 的 predicate

`Objects.nonNull(obj)`

回傳 `true` 若 `obj ≠ null`，適合作為 Stream 的 predicate

```
1 List<String> list = Arrays.asList("Java", null, "Python");
2 list.stream().filter(Objects::nonNull)
3     .forEach(System.out::println); // Java, Python
4 System.out.println(Objects.isNull(null)); // true
```


Objects.requireNonNullElse()

方法

說明

`requireNonNullElse(obj, default)`

若 `obj` 為 null 則回傳 `default`

`requireNonNullElseGet(obj, supplier)`

若 `obj` 為 null 則呼叫 `supplier` (延遲求值)

```
1 String name = Objects.requireNonNullElse(input, "訪客");
2 // input 若為 null 則 name = "訪客"
3
4 String v = Objects.requireNonNullElseGet(
5     cached, () → loadFromDB()); // 只在 null 時才呼叫
```

哈希碼與 hashCode()

什麼是哈希碼？

- **Hash (雜湊)** 源自一位數學家的名字，他發明了雜湊演算法
- 主要目的：**在集合中提升搜尋效率**
- **hashCode()** 根據演算法將物件資訊映射成一個**整數 (雜湊碼)**
- 不同 JVM 的實作與記憶體位址有關，但不保證就是位址本身

💡 **說明：** 雜湊碼有時也稱「散列值」(hash code / hash value)

hashCode() 基本規則

情況	結果
相同的方法 + 相同的值	傳回 相同 hash 值
不同的方法 + 相同的值	傳回 不同 hash 值
不同物件 (預設 Object)	通常傳回 不同 hash 值

hashCode() – 初探

同一個類別，相同的字串值 → 相同的雜湊碼：

```
1  String str1 = "Foo";  
2  String str2 = "Foo";  
3  System.out.println(str1.hashCode()); // 70822  
4  System.out.println(str2.hashCode()); // 70822
```

💡 說明：str1 與 str2 都呼叫 String 的 hashCode()，值相同，所以結果一致

hashCode() – 不同類別的比較

不同類別各自定義 `hashCode()`，相同「值」也可能得出營結果：

```
1  String str = "Foo";  
2  Integer intObj = 10;  
3  
4  System.out.println(str.hashCode());    // 70822  
5  System.out.println(intObj.hashCode()); // 10
```

💡 說明：String 與 Integer 各自 Override 了 hashCode()，使用不同的演算法

現代化的 hashCode() 實作

現代開發不再手寫複雜的雜湊演算法，而是使用 `Objects.hash()` 。

方法

說明

`Objects.hash(v1, v2 ... )`

傳入多個欄位，自動計算出高品質的雜湊碼

```
1  import java.util.Objects;
2
3  class User {
4      String id;
5      String email;
6
7      @Override
8      public int hashCode() {
9          // 傳入所有用於判斷相等的欄位
10         return Objects.hash(id, email);
11     }
12 }
```


equals() 方法

`=` 與 `equals()` 的差異

比較方式

說明

`=` 運算子

比較參照是否指向同一物件

`equals()` 方法

比較物件內容是否相同 (可 Override)

```
1 String s1 = new String("Java");
2 String s2 = new String("Java");
3 System.out.println(s1 == s2); // false (不同物件)
4 System.out.println(s1.equals(s2)); // true (內容相同)
```

💡 字串比對請務必用 `equals()` ; `=` 只判斷是否為同一個物件

Object 的 equals() – 參照比較

Object 原生的 **equals()** 比較的是參照 (reference) 是否指向同一個物件：

```
1  Animal a1 = new Animal("Foo");  
2  Animal a2 = new Animal("Foo");  
3  System.out.println(a1.equals(a2)); // false
```

```
1  Animal a3 = a1;  
2  System.out.println(a1.equals(a3)); // true
```

String 的 equals() – 內容比較

String 已 Override **equals()**，改為比較字串內容是否相同：

```
1 String s1 = "Foo";
2 String s2 = "Foo";
3 System.out.println(s1.equals(s2)); // true
```

類別	equals() 比較對象	範例結果
Object (預設)	參照位址	不同物件 → false
String (Override)	字串內容	內容相同 → true

現代化的 equals() 實作 (JDK 16+)

JDK 16 引入 **Pattern Matching** 後，實作 `equals()` 變得更簡潔且易讀。

```
1  @Override
2  public boolean equals(Object o) {
3      // 1. 同一參照直接回傳 true
4      if (this == o) return true;
5
6      // 2. 判斷型別並同時宣告變數 (JDK 16 Pattern Matching)
7      if (o instanceof User other) {
8          // 3. 比較各欄位內容 (使用 Objects.equals 避免 null)
9          return Objects.equals(this.id, other.id) &&
10             Objects.equals(this.email, other.email);
11     }
12     return false;
13 }
```

💡 重要：若 Override 了 `equals()`，必須同時 Override `hashCode()` ！

equals() 與 hashCode() 的合約

規則	說明
相等必須相同 hash	<code>a.equals(b)</code> 為 <code>true</code> → <code>a.hashCode() = b.hashCode()</code>
相同 hash 不必相等	允許碰撞 (hash 相同但 <code>equals()</code> 不一定為 true)
Override 連動	覆寫 <code>equals()</code> 必須同時覆寫 <code>hashCode()</code>

```
1  User u1 = new User("alice"), u2 = new User("alice");
2  System.out.println(u1.equals(u2));    // true
3  Set<User> set = new HashSet<>();
4  set.add(u1);
5  System.out.println(set.contains(u2)); // false ! hashCode 未 Override
```

toString() 方法

Object 的 toString() – 預設格式

`Object.toString()` 預設回傳格式：類別名稱@hash值（十六進位）

```
1  Animal a = new Animal();
2  System.out.println(a.toString()); // Animal@1b6d3586
3  System.out.println(a);           // 也會自動呼叫 toString()
```

💡 說明：預設格式對使用者不具可讀性，通常需要 Override 成有意義的字串

Override toString() – 範例

```
1  class Animal {  
2      String name = "Foo";  
3      int age = 1;  
4      public String toString() {  
5          return "Name: " + name + ", Age: " + age;  
6      }  
7  }
```

```
1  Animal a = new Animal();  
2  System.out.println(a); // Name: Foo, Age: 1
```


Records 與 Object 方法

紀錄類別 (Records) 與 Object 方法

JDK 16+ 的 **record** 會自動為你 Override 所有重要的 **Object** 方法。

方法

Record 的預設行為

toString()

顯示類別名與所有屬性值

equals()

比較所有屬性的內容 (State-based)

hashCode()

根據所有屬性產生雜湊值

```
1 // 一行代碼搞定 toString>equals/hashCode
2 record Point(int x, int y) { }
3
4 Point p1 = new Point(10, 20);
5 System.out.println(p1); // Point[x=10, y=20]
```


其他 Object 方法

getClass() – 取得物件的類別

`getClass()` 傳回物件所屬的 `Class` 物件：

```
1  class MyClass { }  
2  
3  MyClass obj = new MyClass();  
4  System.out.println(obj.getClass());  
5  // 輸出：class MyClass
```


getClass() – 常用操作

呼叫方式	說明	輸出
<code>obj.getClass()</code>	傳回 Class 物件	<code>class MyClass</code>
<code>obj.getClass().getName()</code>	取得類別名稱字串	<code>"MyClass"</code>

```
1  MyClass obj = new MyClass();
2  System.out.println(obj.getClass());
3  System.out.println(obj.getClass().getName());
```

clone() 方法與 Cloneable 介面

概念

說明

Cloneable

標記介面（無方法），表示允許複製

淺層複製

基本型態欄位複製值；物件欄位複製[參照](#)

深層複製

連物件欄位也複製，兩份完全獨立

```
1  class Point implements Cloneable {  
2      int x, y;  
3      public Point clone() throws CloneNotSupportedException {  
4          return (Point) super.clone();  
5      }  
6  }
```


clone() – 淺層複製的陷阱

```
1 // Owner 有 pet 欄位 (物件型態)
2 Owner o1 = new Owner(); o1.pet = new Pet("旺財");
3 Owner o2 = o1.clone(); // 淺層: pet 欄位仍共用
4 System.out.println(o1.pet == o2.pet); // true
5 o2.pet.name = "小白";
6 System.out.println(o1.pet.name); // 小白 (o1 也被改了!)
7 // 深層: 手動重建物件欄位
8 o2.pet = new Pet("旺財"); // 現在 o1, o2 的 pet 各自獨立
```

💡 物件欄位較多時，建議改用複製建構子 (`new Owner(other)`) 取代 `clone()`

finalize() 方法的廢棄 (JDK 9+)

Object 中還有一個 **finalize()** 方法，用於物件被 GC 回收前的清理工作。

- 現況：自 **JDK 9** 起已被標記為 **Deprecated** (廢棄)
- 原因：執行時機不確定、影響效能、可能導致死鎖
- 替代方案：使用 **Try-with-resources** 與 **AutoCloseable** 介面

⚠ 警告：在現代 Java 開發中，絕對不要 Override 或依賴 **finalize()** 方法。

練習：Employee 類別

任務說明

建立一個 **Employee** 類別，包含以下欄位：

```
1  class Employee {  
2      String name;  
3      int age;  
4      String country;  
5  }
```

1. 建立兩個屬性值相同的 **Employee** 物件，比較它們的 **hashCode()**
2. 觀察 **Object** 預設 **hashCode()** 的行為（屬性相同 $\neq$ 相同 hash 碼）
3. 進階：Override **hashCode()**，使屬性相同的物件回傳相同 hash 值

練習：解題提示

1. 建立物件並輸出 hashCode

- 以 `new Employee( ... )` 建立兩次，分別列印 `hashCode()`
- Object 預設 `hashCode()` 基於記憶體位址 → 不同物件結果不同

2. Override hashCode()

```
1  @Override
2  public int hashCode() {
3      return name.hashCode() + age + country.hashCode();
4  }
```

3. 驗證結果：Override 後，屬性相同的物件應回傳相同 hashCode

實戰加料

為期中作業暖身 🔥

實戰：equals() + HashSet 防止重複填寫

問卷規定「同一個 email 不能重複填寫同一張問卷」。正確覆寫 `equals()` + `hashCode()`，讓 `HashSet` 自動幫你擋重複：

```
1  class Submission {
2      String email;
3      int quizId;
4      Submission(String email, int quizId) {
5          this.email = email; this.quizId = quizId;
6      }
7      @Override
8      public boolean equals(Object o) {
9          if (this == o) return true;
10         if (o instanceof Submission other) {
11             return Objects.equals(email, other.email)
12                 && quizId == other.quizId;
13         }
14         return false;
15     }
16     @Override
17     public int hashCode() {
18         return Objects.hash(email, quizId);
19     }
20 }
```


課程結束

下一章見！